



上海科技大学
ShanghaiTech University

Project Practice for Deep Learning

Lecture 01: Introduction

Zhuyu Lin
Spring, 2026

Outline



■ Course Logistics

- ☐ Overall Objective
- ☐ Syllabus / Projects

■ Introduction to Deep Learning

- ☐ Deep Learning and Neural Network Basics
- ☐ Training Neural Network
- ☐ Regularization

Outline



■ Course Logistics

- Overall Objective
- Syllabus / Projects

■ Introduction to Deep Learning

- Deep Learning and Neural Network Basics
- Training Neural Network
- Regularization

Course Objectives



■ Proficiency in Deep Learning (DL)

- ☐ learning to construct, train and evaluate deep neural networks

■ Applied Engineering Competence

- ☐ Gain hands-on experience through real-world projects
- ☐ Develop the ability to address and solve challenges commonly encountered in practical settings.

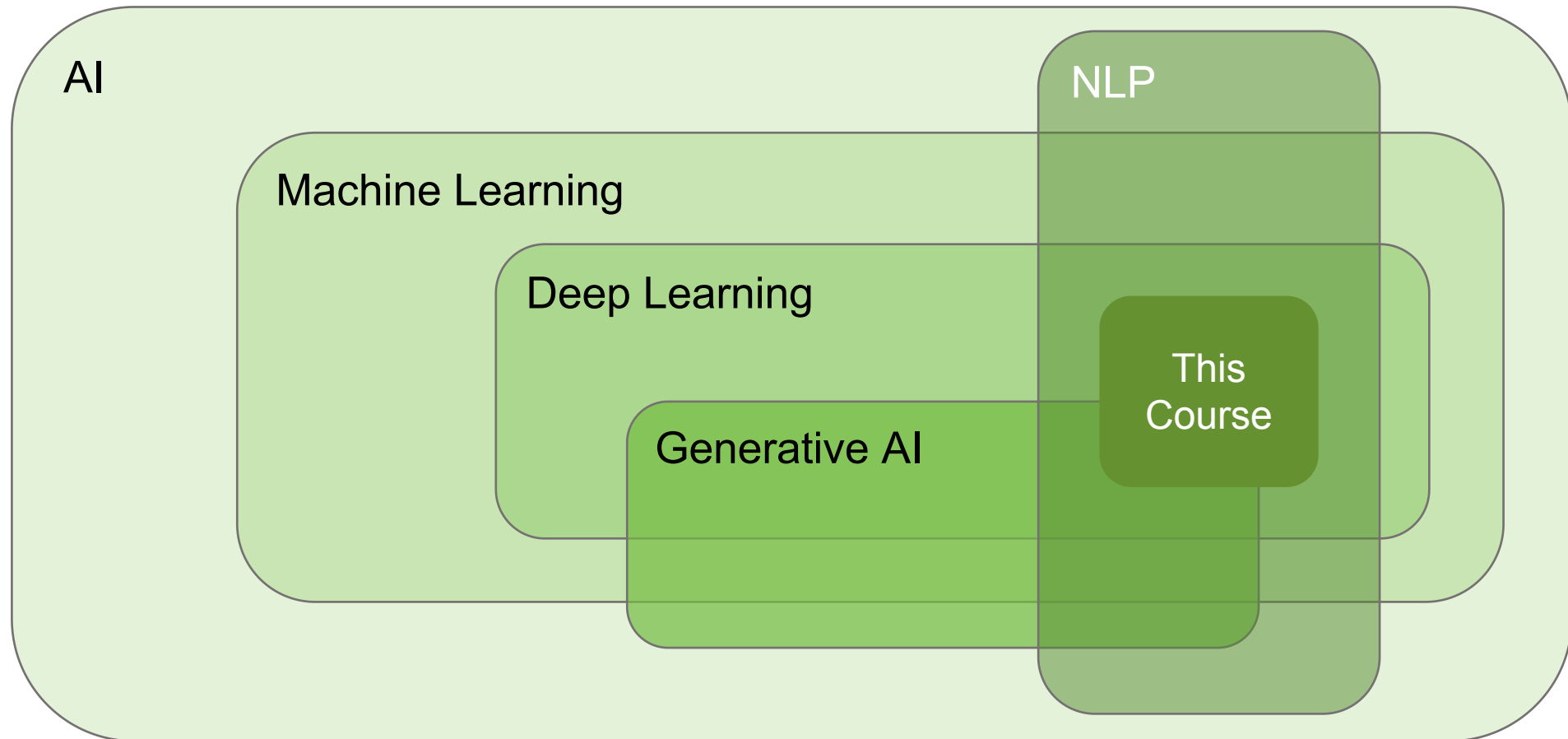
■ Collaborative Skills and Effective Presentation

- ☐ Enhance teamwork capabilities and articulate findings confidently through compelling presentations.

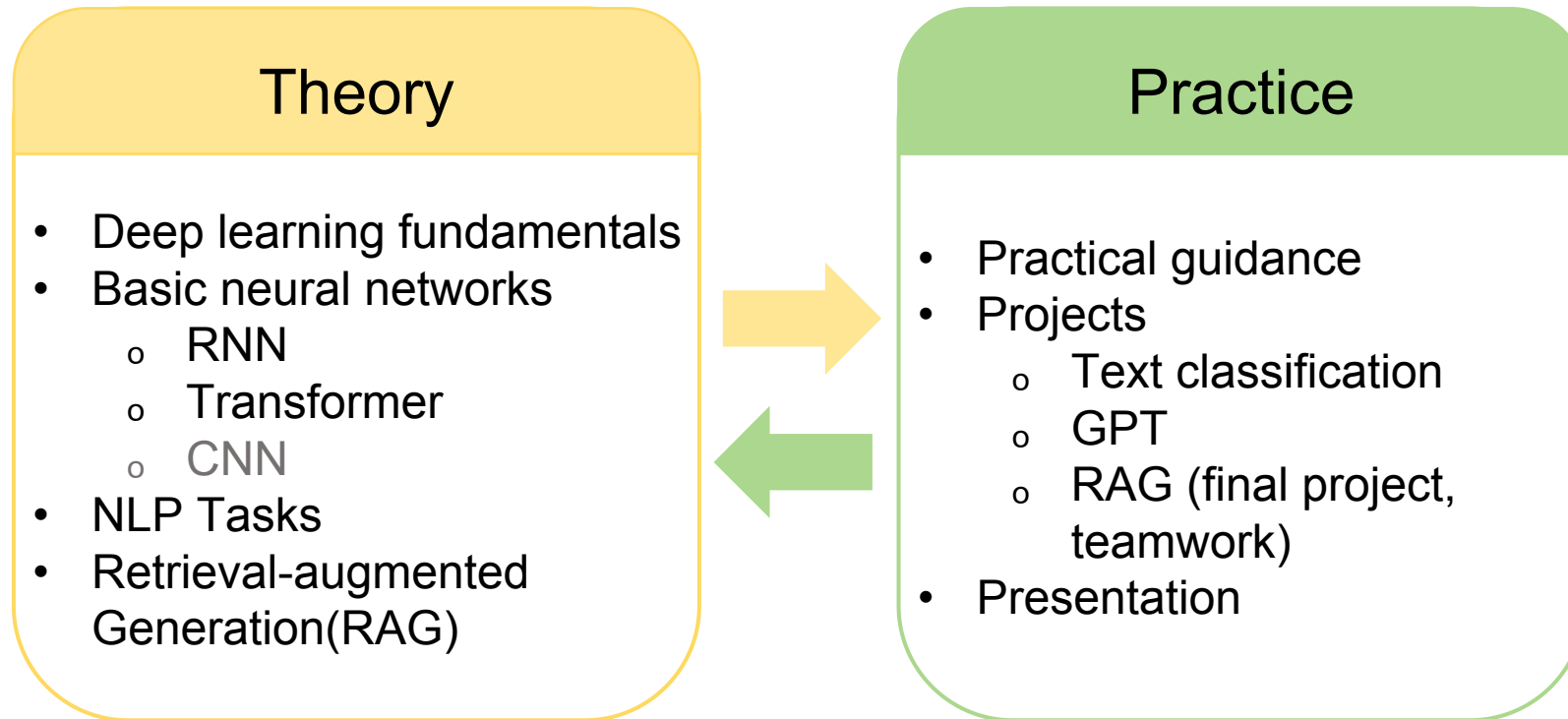
Course Content



上海科技大学
ShanghaiTech University



Course Content



Syllabus & Schedule



Monday, Periods 3 and 4, Weeks 1–16.

三月				四月					五月				六月				
2	9	16	23	30	6	13	20	27	4	11	18	25	1	8	15	22	29
3	10	17	24	31	7	14	21	28	5	12	19	26	2	9	16	23	30
4	11	18	25	1	8	15	22	29	6	13	20	27	3	10	17	24	1
5	12	19	26	2	9	16	23	30	7	14	21	28	4	11	18	25	2
6	13	20	27	3	10	17	24	1劳动节	8	15	22	29	5	12	19端午节	26	3
7	14	21	28	4	11	18	25	2	9	16	23	30	6	13	20	27	4
8	15	22	29	5清明	12	19	26	3	10	17	24	31	7	14	21	28	5
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

春学期

Syllabus & Schedule



上海科技大学
ShanghaiTech University

■ Ecourse platform: <https://ecourse.shanghaitech.edu.cn/>

■ 深度学习工程实践-CS290S.01

The screenshot shows the Ecourse platform interface for the course '深度学习工程实践-CS290S.01'. The top navigation bar includes the ShanghaiTech University logo, 'E云课堂', '公开课', '资源', and a user profile dropdown. The course title is displayed with a '私有' (Private) status. Below the title, there are buttons for '导入课程内容', '互动课堂', and '预览课程'. The main content area features a course description, a list of course navigation items (课程信息, 章节, 公告, 班级成员, 课程信息, 课程题库, 知识库, 课程设置, 邀请), and a '课程导航' (Course Navigation) section with icons for '添加助教', '上传课件', '布置作业', '发布测试', '发布公告', and '知识图谱'. The right sidebar shows '课程公告' (Course Announcements) and '师生互动' (Teacher-Student Interaction) sections, both currently displaying '暂无数据' (No data).

Grading Policy



■ Class Participation: 10%

■ The 1st Project: 25%

- ☐ Work independently
- ☐ 20% basic + 5% advanced

■ The 2nd Project: 30%

- ☐ Work independently
- ☐ 25% basic + 5% advanced

■ Final Project: 35%

- ☐ May work as a group, 2~3 students
- ☐ Final presentation

Reference books and materials



上海科技大学
ShanghaiTech University

■ Deep learning:

- <https://d2l.ai/>
- <https://web.stanford.edu/~jurafsky/slp3/>

■ Online deep learning courses:

- Stanford: CS224n, CS231n

■ PyTorch:

- https://docs.pytorch.org/tutorials/beginner/basics/quickstart_tutorial.html

Instructor and TAs



上海科技大学
ShanghaiTech University

■ Instructor: Zhuyu Lin

- linzhy2@shanghaitech.edu.cn
- SIST 1B206

■ TAs:

- Lanzhuju Mei, meilzhj2023@shanghaitech.edu.cn
- Yiwei Wang, wangyw12024@shanghaitech.edu.cn
- Lei Cui, cuilei2025@shanghaitech.edu.cn

■ Office hours:

- 15:00 – 16:00 Monday
- Please send me an email before coming

Pre-requisite



■ Proficiency in Python

- All class assignments will be in Python (and use PyTorch)

■ Calculus, Linear Algebra, Probability and Statistics

- Undergrad course level

■ Machine Learning Fundamentals

- Formulating cost functions
- Taking derivatives
- Performing optimization with gradient descent

Outline



■ Course Logistics

- Overall Objective
- Syllabus / Projects

■ Introduction to Deep Learning

- Deep Learning and Neural Network Basics
- Training Neural Network
- Regularization

Introduction



■ Goal of AI: build intelligent algorithms

□ to understand data

Sentiment analysis

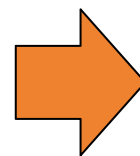
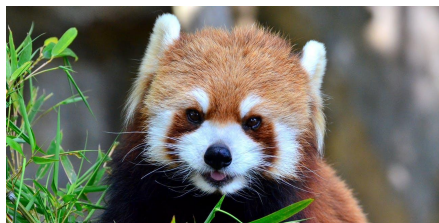


这部电影太精彩了! 剧情紧凑, 演员演技在线, 强烈推荐!



对白生硬, 情节拖沓, 根本撑不起两小时片长。

Recognizing objects
in images



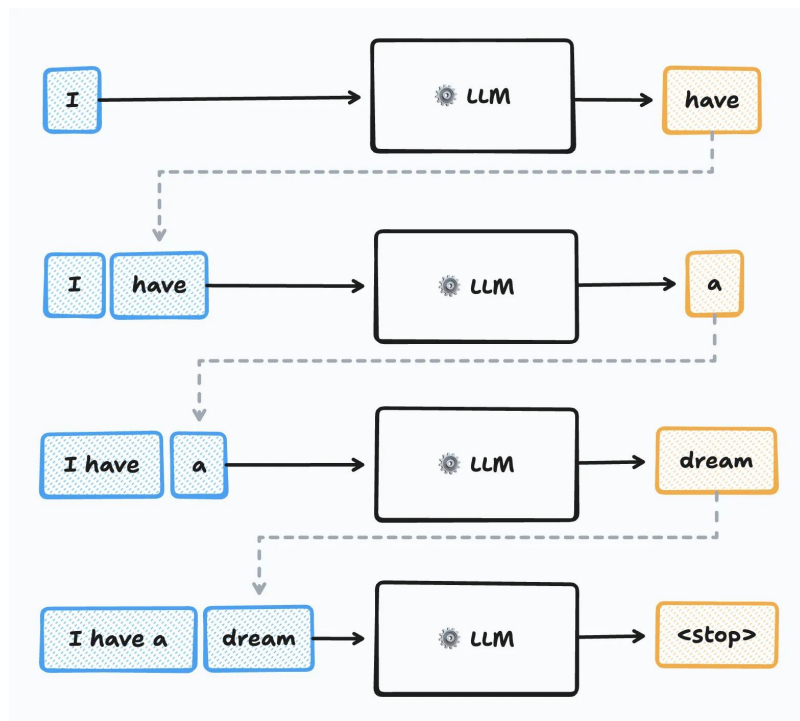
red panda (*Ailurus fulgens*)

Introduction



■ Goal of AI: build intelligent algorithms

- to understand data
- to generate data



<https://pm.dartus.fr/posts/2025/how-llm-generate-text/>



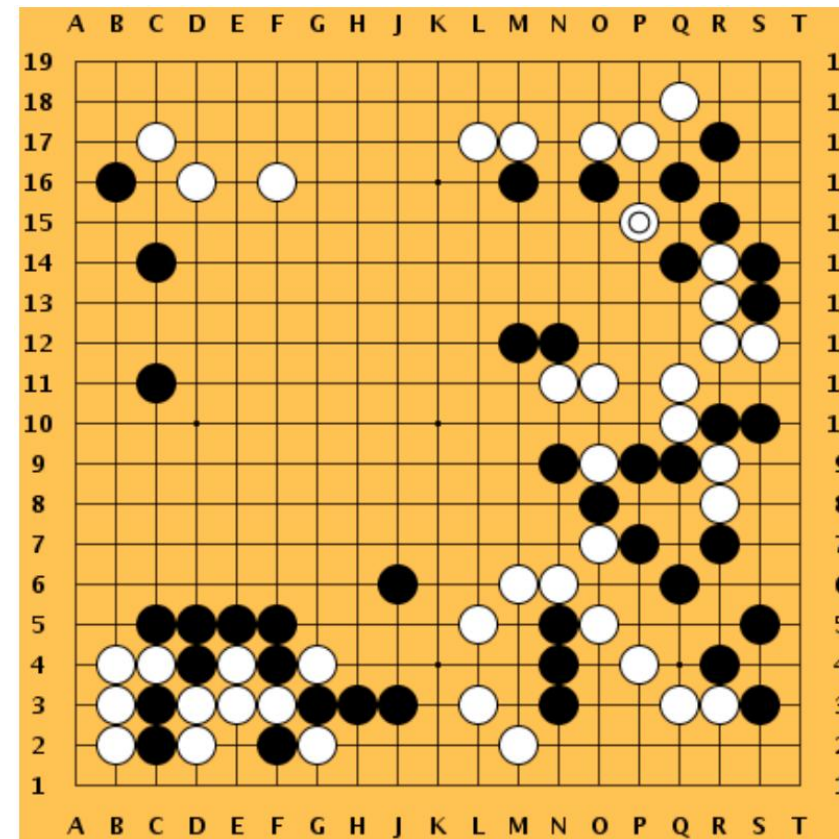
<https://dalle3.ai/>

Introduction



■ Goal of AI: build intelligent algorithms

- to understand data
- to generate data
- to make decisions



Introduction



■ Main Branches of AI (by modality)

Natural
Language
Processing
(NLP)

Computer
Vision
(CV)

Speech
Processing

Multimodal

Embodied
Intelligence

Deep Learning Applications



■ A broad range of real-world applications

□ Speech recognition

■ Input: sound wave → Output: transcript

□ Language translation

■ Input: text in language A (Eng) → Output: text in language B (Chs)

□ Image classification

■ Input: images → Output: image category (cat, dog, car, house, etc.)

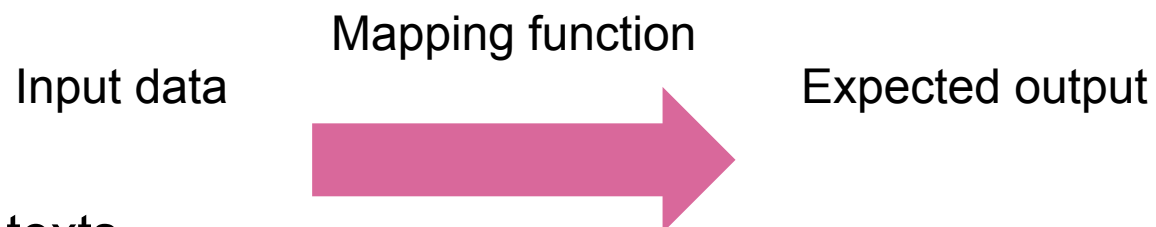
□ Autonomous driving

■ Input: sensory inputs → Output: actions (straight, left, right, stop, etc.)

Main challenges: difficult to manually design the algorithms

A data-driven approach

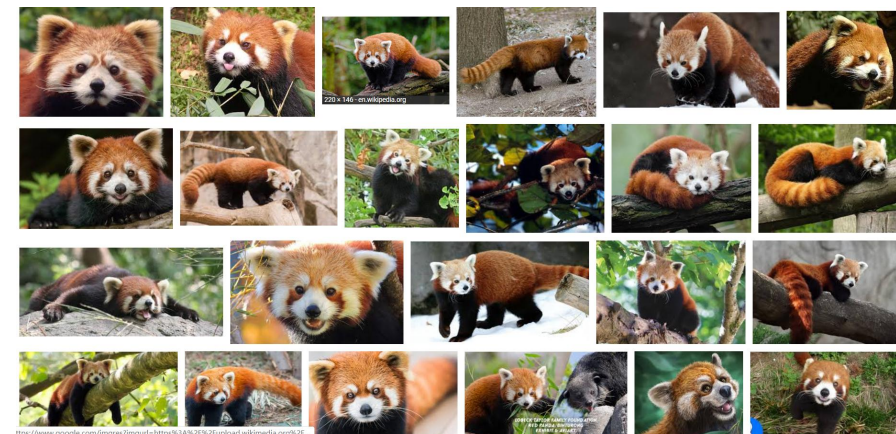
■ Each task is to obtain a mapping function (or a model)



- input data: images / texts
- expected output: category

■ Building such mapping functions from data

text	label
虽然是小成本制作，但情感真挚，细节打动人心。	1
对白生硬，情节拖沓，根本撑不起两小时片长。	0
这部电影太精彩了！剧情紧凑，演员演技在线，强烈推荐！	1
节奏慢得像蜗牛，两个小时都在打瞌睡。	0



red panda (*Ailurus fulgens*)



A data-driven approach

■ Building a **mapping function** (model)

$$y = f(x; \theta)$$

- x : input data
- y : expected output
- θ : parameters to be estimated

■ **Learning** the model from data

- Given a dataset $D = \{(x_n, y_n)\}_{n=1}^N$
- Find the 'best' parameter $\tilde{\theta}$, such that

$$y_i \cong f(x_i; \tilde{\theta})$$

- And it can be generalized to unseen input data

A Probabilistic View



- The joint likelihood of all the samples in the dataset D are modeled as

$$P((x_1, y_1), \dots, (x_N, y_N)) = \prod_{i=1}^N P(x_i, y_i) = \prod_{i=1}^N P(y_i|x_i)P(x_i)$$

- Among them, prediction as a functional mapping from the feature to the outcome.

$$P(y|x) = f(y|x; \theta)$$

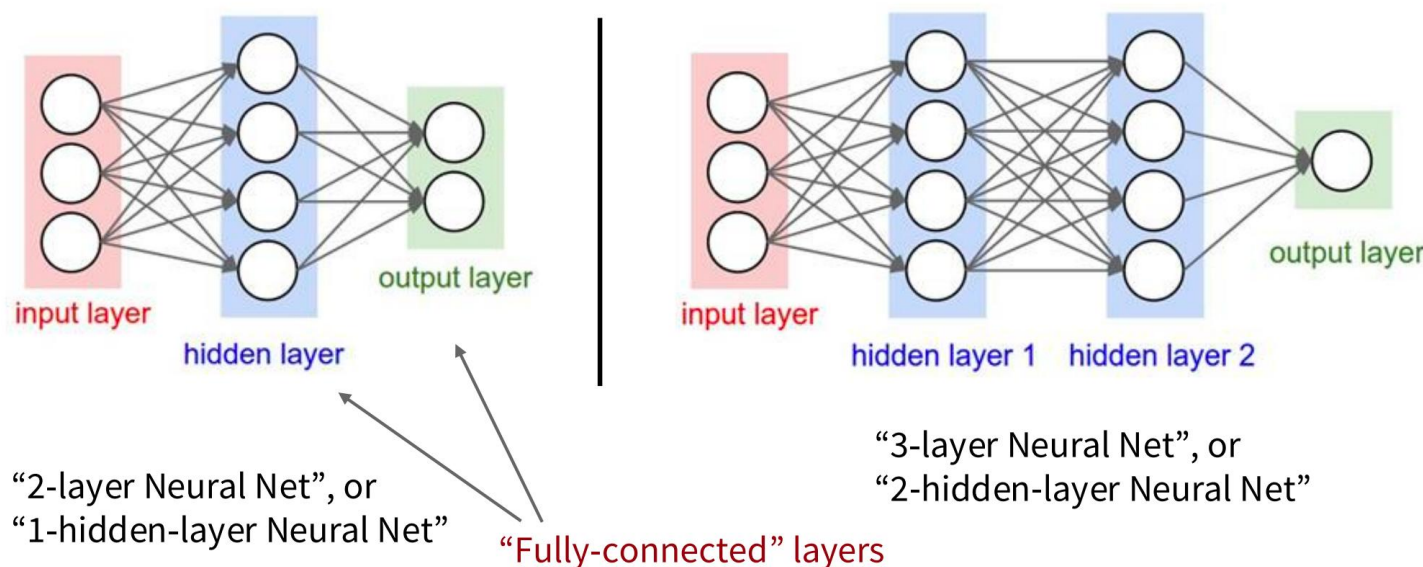
- And the probability of the sample $P(x_i)$ can be discarded from the view of maximum likelihood estimation, i.e.,

$$\begin{aligned}\tilde{\theta} &= \operatorname{argmax}_{\theta} \log \prod_{i=1}^N P(y_i|x_i)P(x_i) \\ &= \operatorname{argmin}_{\theta} - \left[\sum_{i=1}^N \log P(y_i|x_i) + \sum_{i=1}^N \log P(x_i) \right]\end{aligned}$$

Neural Networks

■ Using neural networks as the mapping function

- 2-layer Neural Network: $f = W_2 \max(0, W_1 x)$, $x \in R^D$, $W_1 \in R^{H \times D}$, $W_2 \in R^{K \times H}$
- 3-layer Neural Network: $f = W_3 \max(0, W_2 \max(0, W_1 x))$, $x \in R^D$, $W_1 \in R^{H_1 \times D}$, $W_2 \in R^{H_2 \times H_1}$, $W_3 \in R^{K \times H_2}$

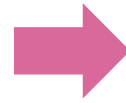


- “Neural Network” is a very broad term; these are more accurately called “**fully-connected networks**” or sometimes “**multi-layer perceptrons**” (MLP)
- In practice we will usually add a **learnable bias** at each layer as well: $f = W_2 \max(0, W_1 x + b_1) + b_2$

Activation Functions



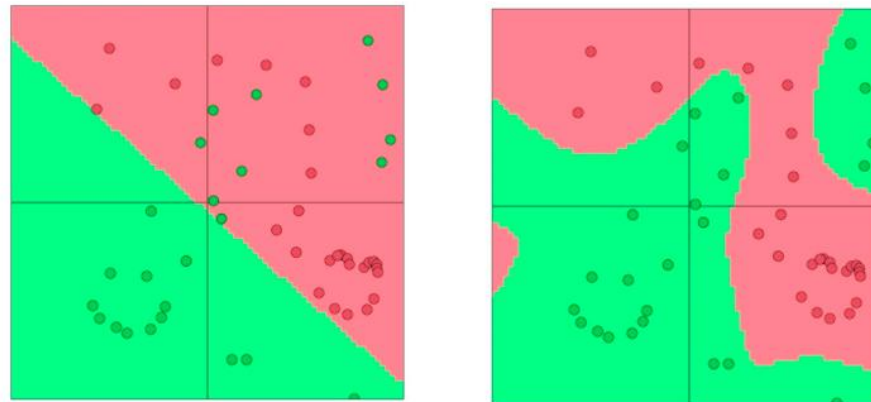
$$f = W_2 \max(0, W_1 x)$$



The function $\max(0, z)$ is called the **activation function**

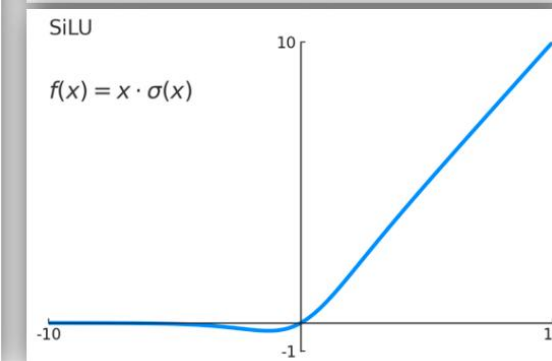
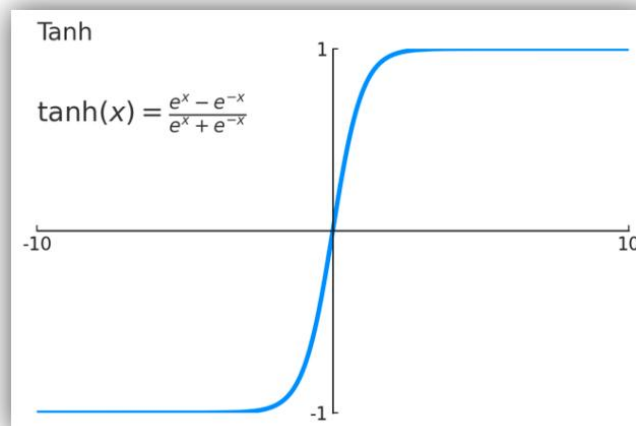
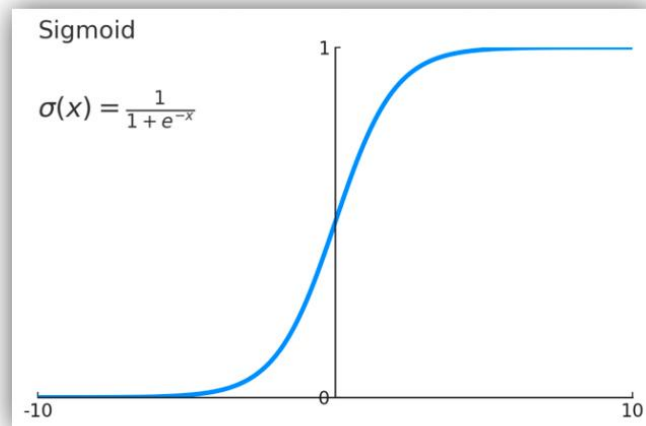
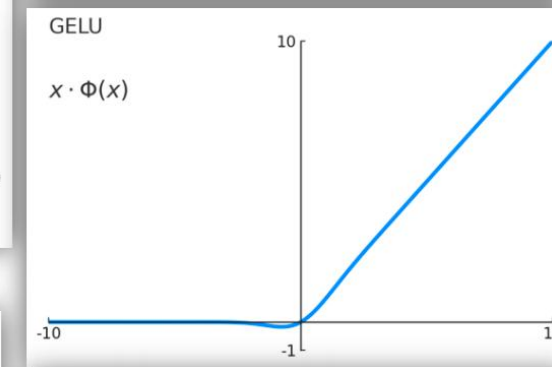
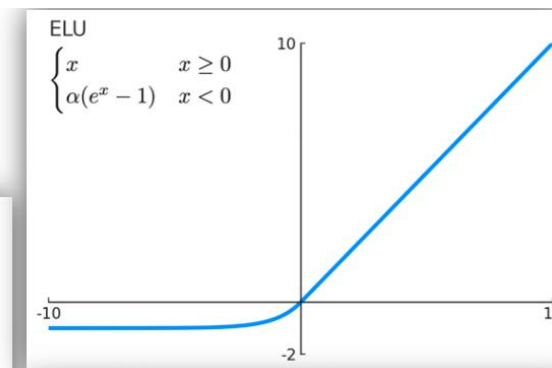
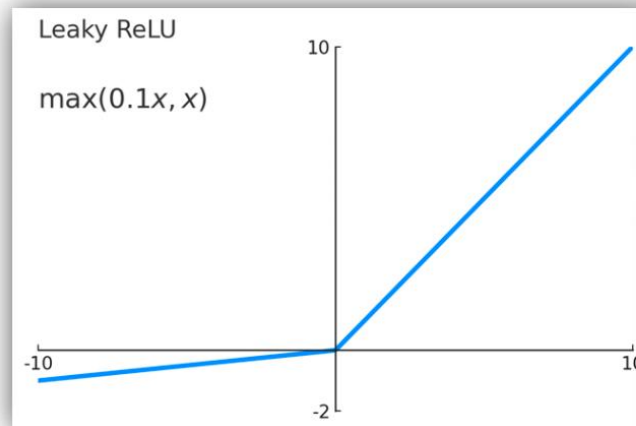
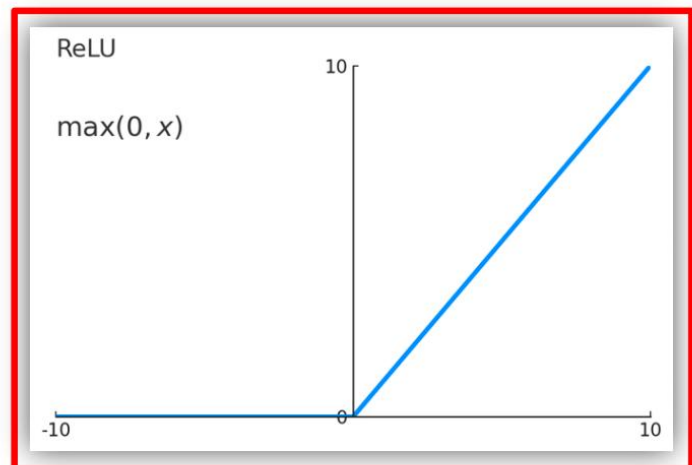
■ Activation function introduce **Non-linearity** to neural networks.

- Without non-linearities, deep neural networks can't do anything more than a linear transform.
- Extra layers could just be compiled down into a single linear transform: $W_1 W_2 x = Wx$
- But, with more layers that include non-linearities, they can approximate more complex functions!

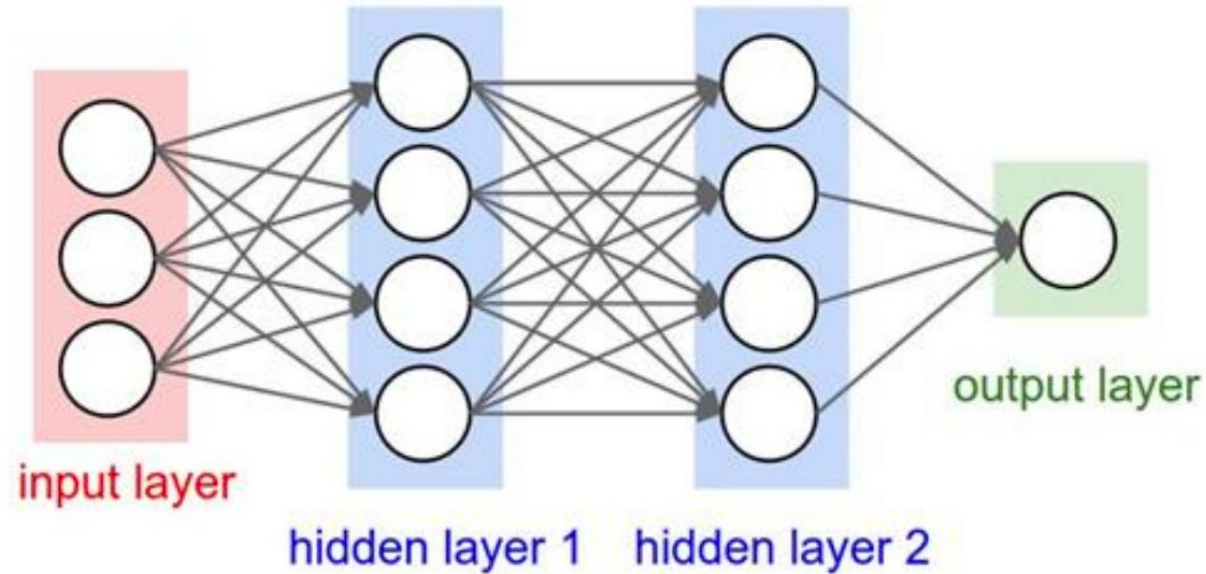


Activation functions

ReLU is a good default choice for most problems



Neural Networks



```
# forward-pass of a 3-layer neural network:  
f = lambda x: 1.0/(1.0 + np.exp(-x)) # activation function (use sigmoid)  
x = np.random.randn(3, 1) # random input vector of three numbers (3x1)  
h1 = f(np.dot(W1, x) + b1) # calculate first hidden layer activations (4x1)  
h2 = f(np.dot(W2, h1) + b2) # calculate second hidden layer activations (4x1)  
out = np.dot(W3, h2) + b3 # output neuron (1x1)
```

Neural Networks



pytorch implementation

```
import torch
import torch.nn as nn

class ThreeLayerMLP(nn.Module):
    def __init__(self, input_size, hidden_size1, hidden_size2, output_size):
        super(ThreeLayerMLP, self).__init__()

        # Define the three linear layers
        self.fc1 = nn.Linear(input_size, hidden_size1)
        self.fc2 = nn.Linear(hidden_size1, hidden_size2)
        self.fc3 = nn.Linear(hidden_size2, output_size)

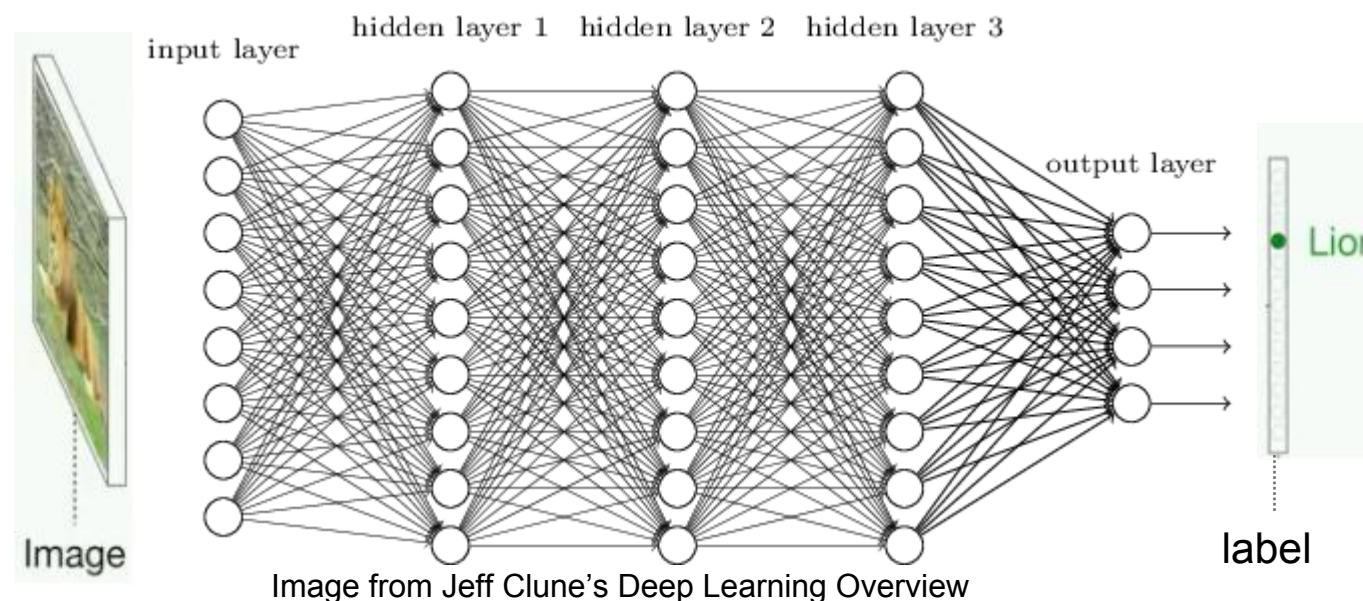
        # Activation function(use relu)
        self.relu = nn.ReLU()

    def forward(self, x):
        out = self.fc1(x)
        out = self.relu(out)
        out = self.fc2(out)
        out = self.relu(out)
        logits = self.fc3(out)
        return logits
```

What is deep learning?

■ Deep: Deep neural networks

- Constructing a **multi-layer** representation of input



What is deep learning?

■ Deep: Deep neural networks

- Constructing a **multi-layer** representation of input

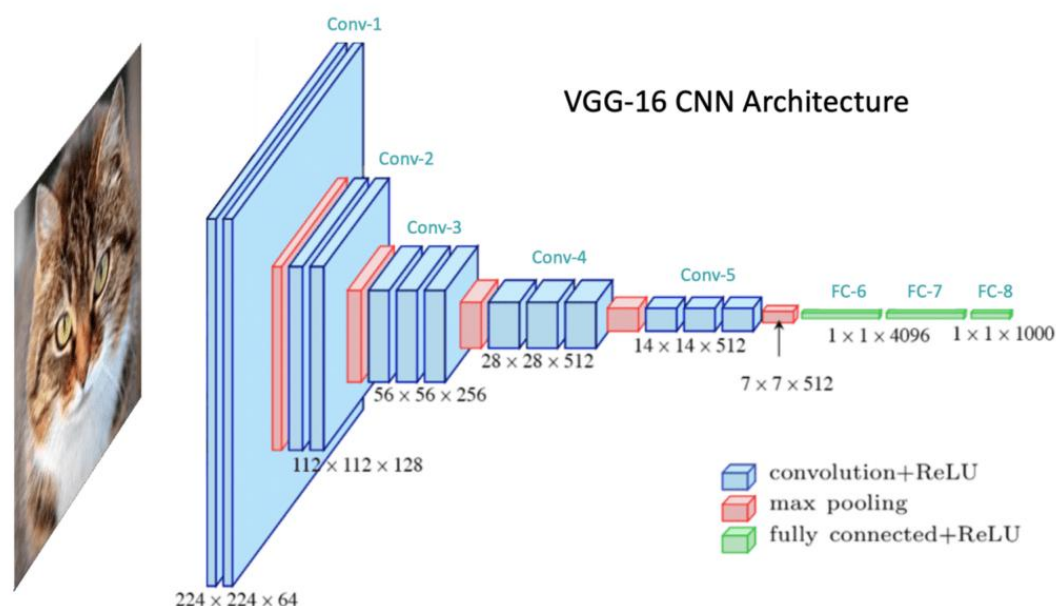


image source: <https://learnopencv.com/understanding-convolutional-neural-networks-cnn/>

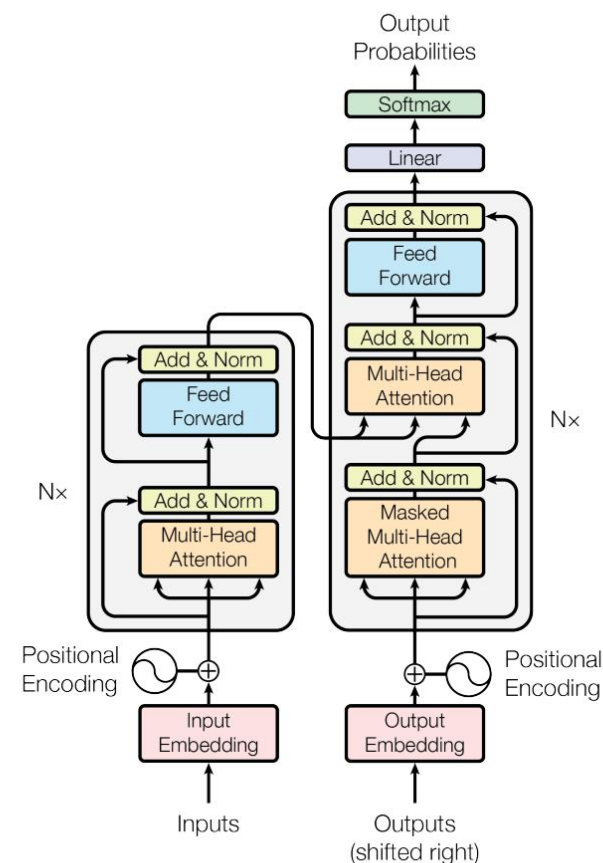


Figure 1: The Transformer - model architecture.

<https://arxiv.org/abs/1706.03762>

What is deep learning?



■ Learning: Parameter estimation from data

- Parameters: connection weights between units
- Formulated as an optimization problem
- Efficient algorithms for handling large-scale models & datasets

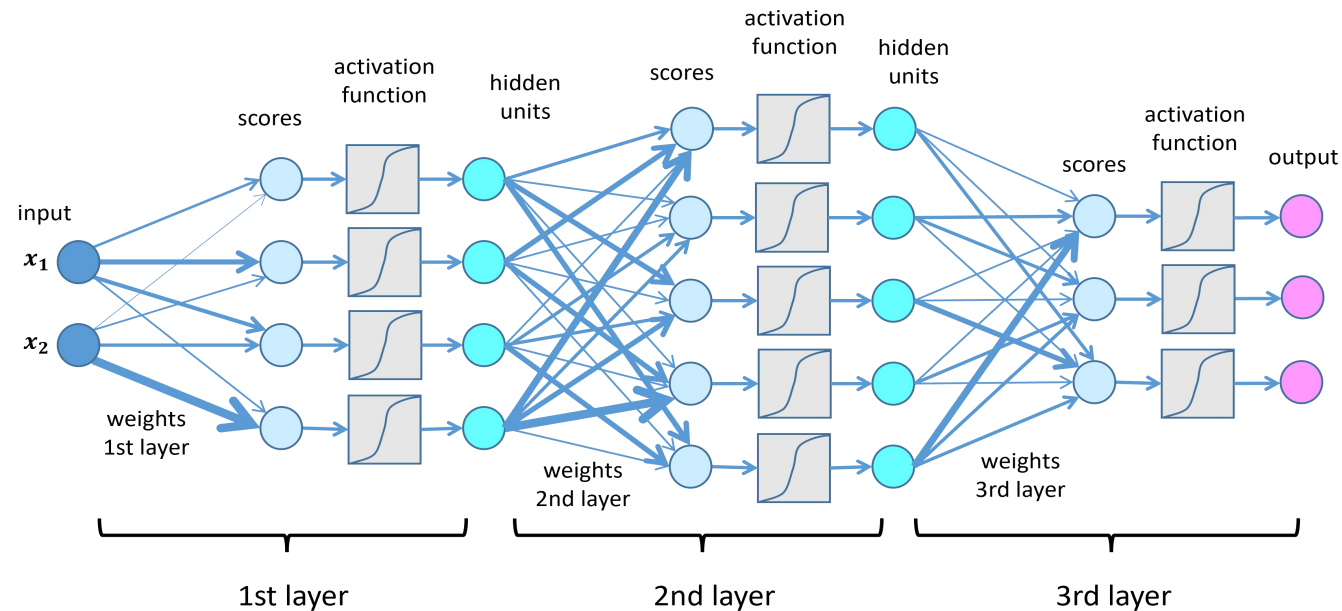
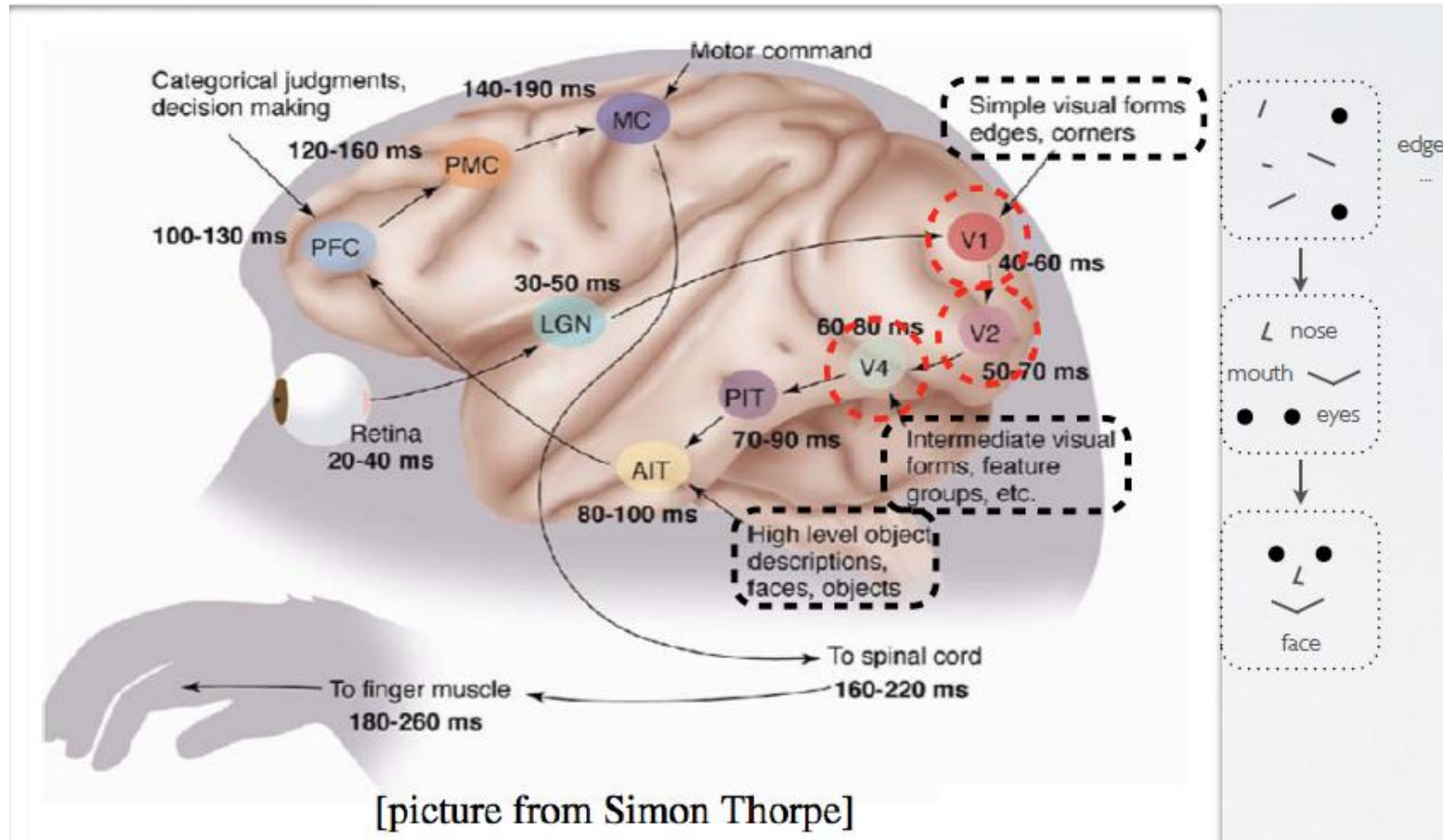


image source: <https://lamarr-institute.org/blog/deep-neural-networks/>

Why deep networks?

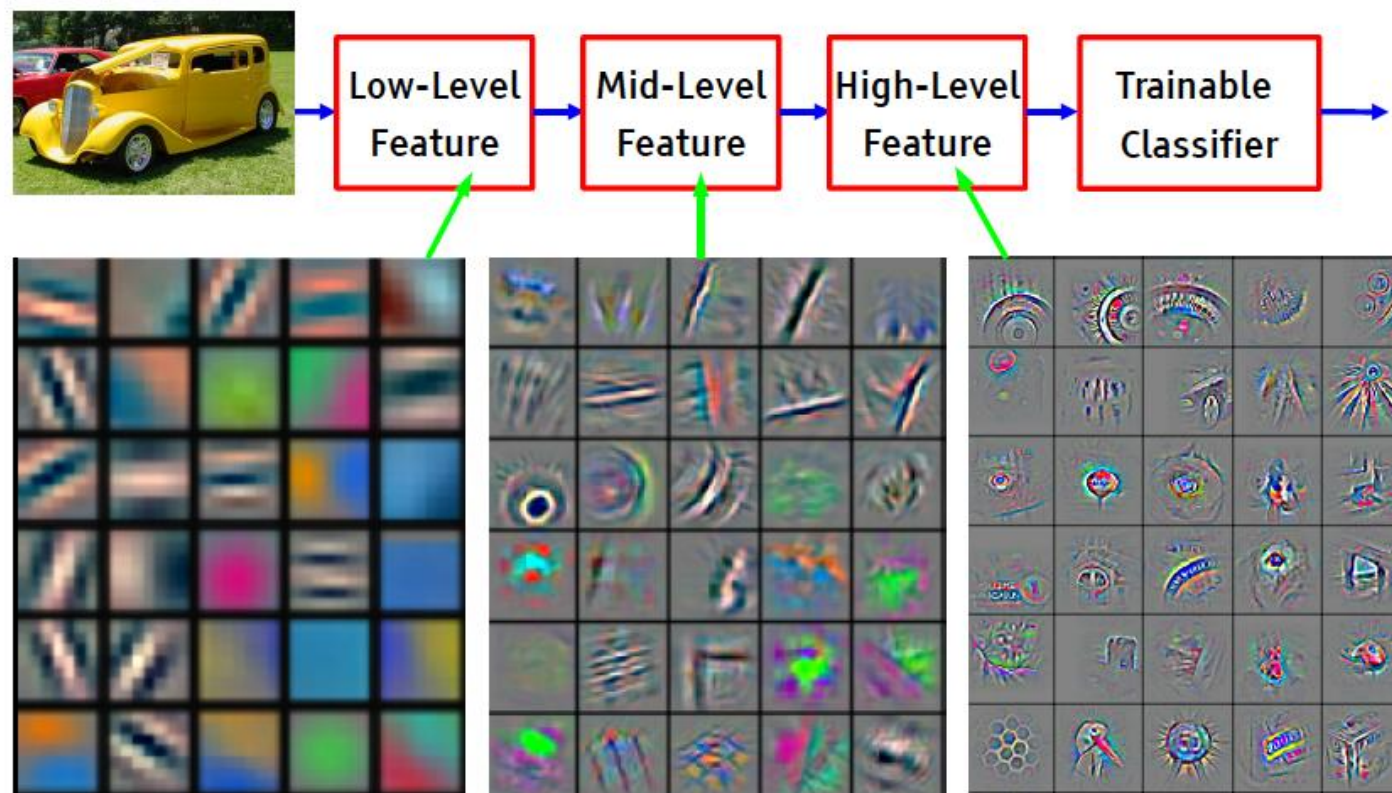


■ Inspiration from visual cortex



Why deep networks?

- **Hierarchical Feature Learning:** In deep networks, each layer learns to extract features at different levels of abstraction.



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

Why deep networks?



- **Hierarchical Feature Learning:** In deep networks, each layer learns to extract features at different levels of abstraction.
- **Increased Capacity:** More layers mean the network can model more complex functions.
- **Representation Power:** A deep network can represent certain functions (exponentially) more compactly than shallow networks.

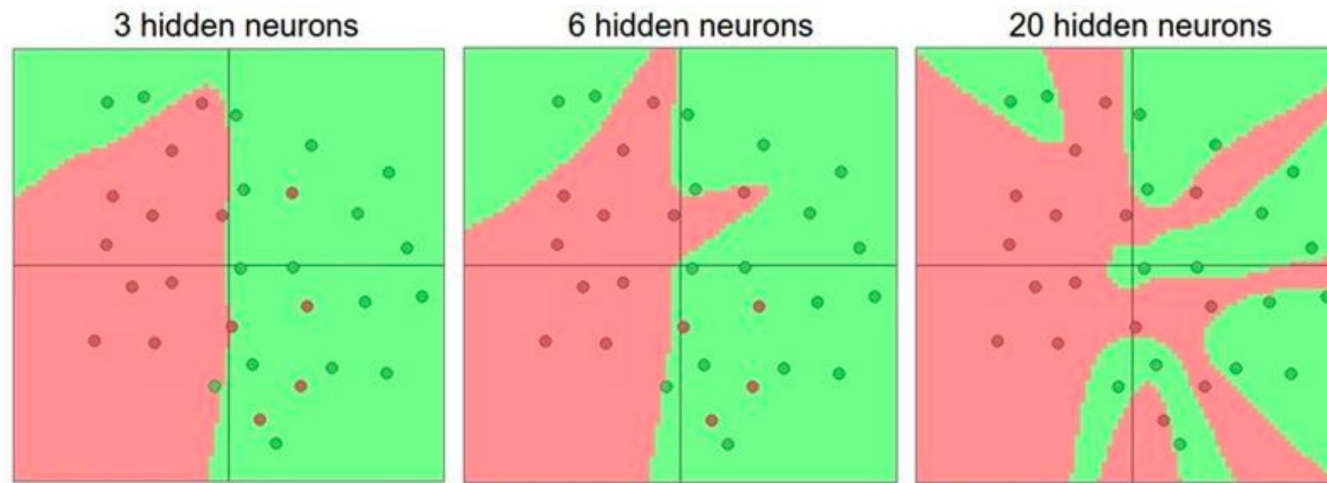


image source: https://cs231n.stanford.edu/slides/2025/lecture_4.pdf

Outline



■ Course Logistics

- Overall Objective
- Syllabus / Projects

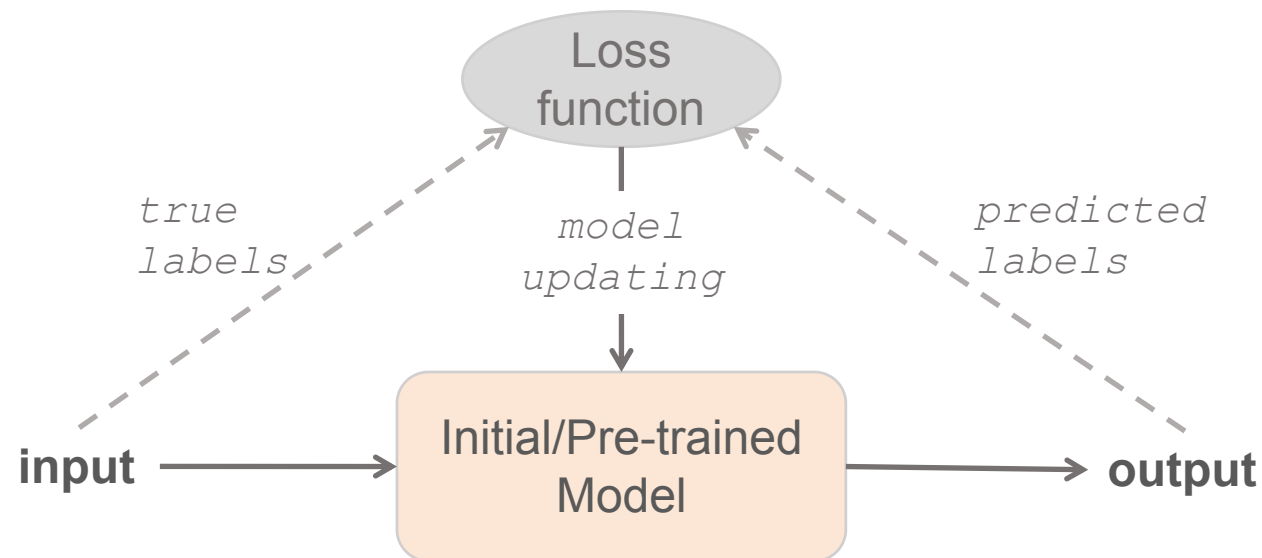
■ Introduction to Deep Learning

- Deep Learning and Neural Network Basics
- Training Neural Network
- Regularization

Training Neural Networks



- Define a **loss function** $L(y_{\text{pred}}, y_{\text{gt}})$ that quantifies the error between the predicted output and the ground truth.
- Find the best W that **minimizes the loss** using **optimization algorithms**





Loss Function

■ Cross-entropy Loss (for classification)

- Cross-entropy loss for the **target probability** distribution $p^* = (0, \dots, 0, 1, 0, \dots)$ (1 for the target label, 0 for the rest) and the **predicted by the model distribution** $p = (p_1, \dots, p_K)$, $p_i \in (0, 1)$, $\sum_i^K p_i = 1$:

$$H(p^*, p) = - \sum_{i=1}^K p_i^* \log(p_i)$$

- Since only one of p_i^* is non-zero (1 for the target label k , 0 for the rest), we will get

$$H(p^*, p) = -\log(p_k)$$

- i.e. the only term left as our loss function is the negative log probability of the true class y_i :

$$L_i = -\log P(y_i | x_i)$$
$$L = \frac{1}{N} \sum_i^N L_i = -\frac{1}{N} \sum_i^N \log P(y_i | x_i)$$



Loss Function

- Cross-entropy Loss (for classification)

$$L = -\frac{1}{N} \sum_i^N \log P(y_i | x_i)$$

Use this in PyTorch!
`torch.nn.CrossEntropyLoss()`

- We usually **softmax** transforms raw model outputs(logits) into a probability distribution $p = (p_1, \dots, p_K)$, $p_i \in (0, 1)$, $\sum_i^K p_i = 1$.
- For a vector $\mathbf{z}$ of K real numbers $\mathbf{z} = [z_1, z_2, \dots, z_K]$, the softmax function is calculated as:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j^K e^{z_j}}$$

- `torch.nn.CrossEntropyLoss()` combines **LogSoftmax** and **NLLLoss** (Negative Log Likelihood Loss) to compute the loss between **input logits** and **target labels**.

Optimization: Gradient Descent

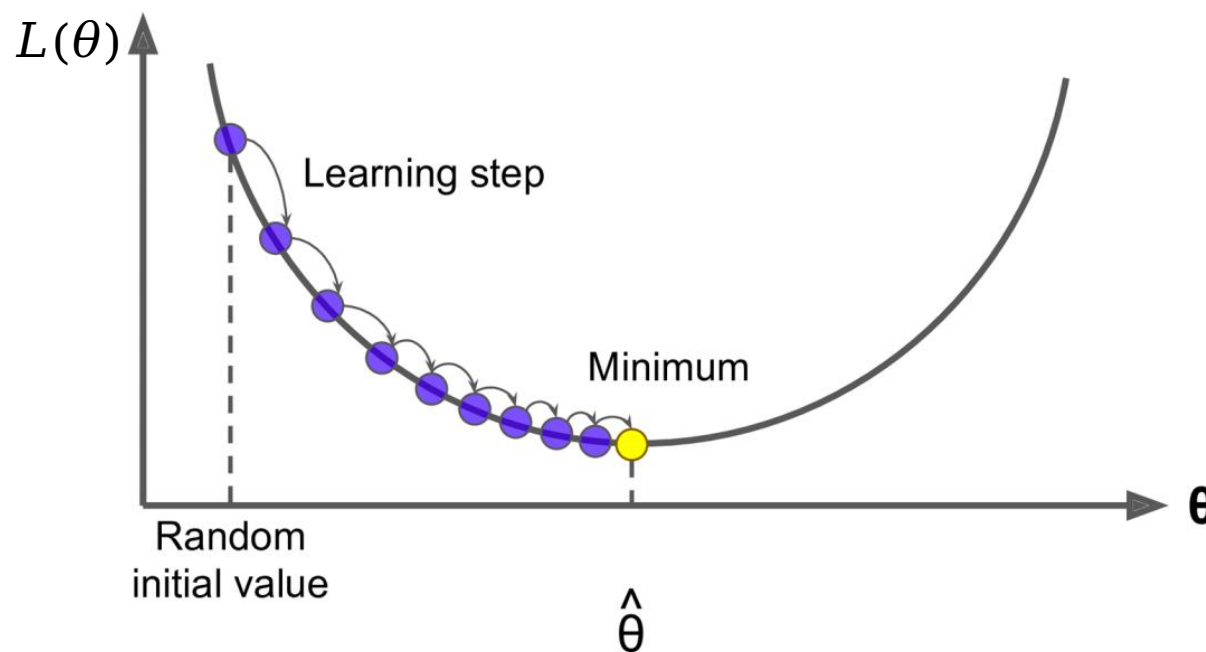


$$P(y|x) = f = \text{softmax}(W_2 \max(0, W_1 x + b_1) + b_2)$$

$$L(\theta) = -\frac{1}{N} \sum_i \log P(y_i|x_i)$$

θ represents all the model parameters (W, b)

■ Gradient Descent is an algorithm to minimize $L(\theta)$



Optimization: Gradient Descent



- Update equation (in matrix notation):

$$\theta^{new} = \theta^{old} - \alpha \nabla_{\theta} L(\theta)$$

α = step size or learning rate

- Update equation (for each single parameter θ):

$$\theta_j^{new} = \theta_j^{old} - \alpha \frac{\partial}{\partial \theta_j^{old}} L(\theta)$$

- Algorithm:

```
# Vanilla Gradient Descent
```

```
while True:
```

```
    weights_grad = evaluate_gradient(loss_fun, data, weights)
```

```
    weights += - step_size * weights_grad # perform parameter update
```

Stochastic Gradient Descent (SGD)



上海科技大学
ShanghaiTech University

$$L(\theta) = -\frac{1}{N} \sum_i^N \log P(y_i | x_i)$$

- Problem: Full sum is expensive when N is large!
- Solution: Stochastic Gradient Descent (SGD)
 - Approximate sum using a minibatch of examples 32 / 64 / 128 / 256
 - Repeatedly sample mini batches, and update after each one
- Algorithm:

```
# Vanilla Minibatch Gradient Descent
```

```
while True:
```

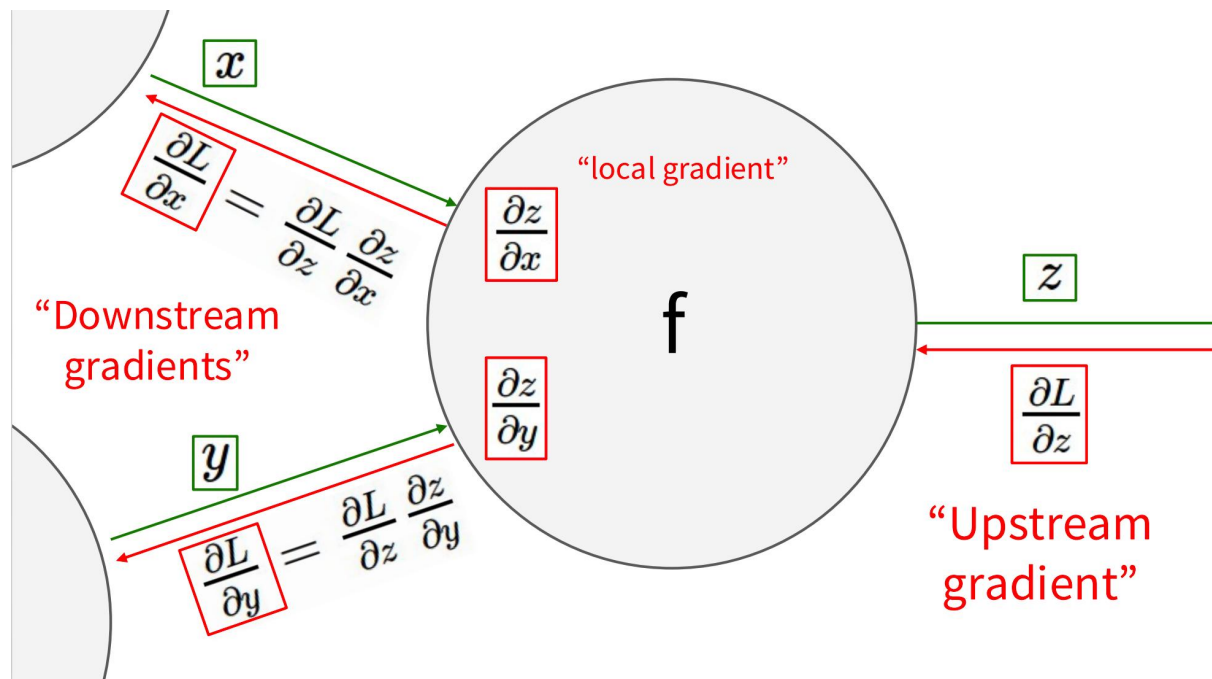
```
    data_batch = sample_training_data(data, 256) # sample 256 examples  
    weights_grad = evaluate_gradient(loss_fun, data_batch, weights)  
    weights += - step_size * weights_grad # perform parameter update
```


How to compute gradients?



■ **Backpropagation:** recursively (and hence efficiently) apply the chain rule along computation graph

$$\square [\text{downstream gradient}] = [\text{upstream gradient}] \times [\text{local gradient}]$$



<https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1246/slides/cs224n-spr2024-lecture03-neuralnets.pdf>
https://cs231n.stanford.edu/slides/2025/lecture_4.pdf

How to compute gradients?



- **Backpropagation:** recursively (and hence efficiently) apply the chain rule along computation graph
 - $[\text{downstream gradient}] = [\text{upstream gradient}] \times [\text{local gradient}]$
- **Forward pass:** compute results of operations and save intermediate values
- **Backward pass:** apply chain rule to compute gradients
- **Modern deep learning frameworks**(e.g., pytorch) compute gradients for you!

<https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1246/slides/cs224n-spr2024-lecture03-neuralnets.pdf>

Optimization Algorithms



- There are numerous **optimization algorithms** that build upon or enhance basic gradient descent to **improve convergence speed, stability, and performance**.
 - **Momentum**: a method that helps **accelerate** SGD in the right direction and **dampens oscillations**.
 - **Adagrad**: **Adapts the learning rate** for each parameter so that parameters with large gradients get smaller learning rates, and vice versa. However, the learning rate can decay too aggressively and vanish over time.
 - **RMSprop**: Modifies Adagrad to prevent the learning rate from decaying too quickly.
 - **Adam**: Combines the ideas of momentum and RMSprop.
 - **AdamW**: Decouples the weight decay (L2 regularization) from the gradient update step in Adam. This improves **generalization performance** and is particularly **popular in Transformer models**.

suggested readings:

<https://www.ruder.io/optimizing-gradient-descent/>

https://cs231n.stanford.edu/slides/2025/lecture_3.pdf

Training Pipeline



1. Get a dataset of (x, y) pairs
2. Select an appropriate neural network model to formulate the problem as $y=f(x)$
3. Define the loss function
4. Choose an optimizer
5. Initialize the model parameters
6. Train the network iteratively

For each epoch (a full pass over the training data):

- a. **Forward pass:** Feed a batch of inputs through the network to compute predictions and the loss.
- b. **Backward pass (backpropagation):** Compute gradients of the loss with respect to every weight using the chain rule.
- c. **Parameter update:** Adjust the weights using the chosen optimizer and the computed gradients.
- d. Repeat for all batches.

Training Pipeline



■Pytorch code

```
model = ThreeLayerMLP(input_size, hidden_size1, hidden_size2, output_size).to(device)
criterion = nn.CrossEntropyLoss() # Define the loss function
optimizer = AdamW(model.parameters(), lr=learning_rate) # Initialize the optimizer

model.train() # Set the model to training mode
for epoch in range(epochs):
    for batch_x, batch_y in train_loader:
        batch_x, batch_y = batch_x.to(device), batch_y.to(device)

        optimizer.zero_grad() # Clear gradients from the previous step
        # Forward pass
        logits = model(batch_x) # compute predicted logits
        loss = criterion(logits, batch_y) # Compute the loss between predictions and true labels
        # Backward pass
        loss.backward() # compute gradients of the loss with respect to model parameters
        optimizer.step() # Update model parameters using the optimizer
```

Training Pipeline



- As training progresses, the loss gradually decreases, indicating that the model is fitting the training data well and adjusting its parameters to minimize errors.

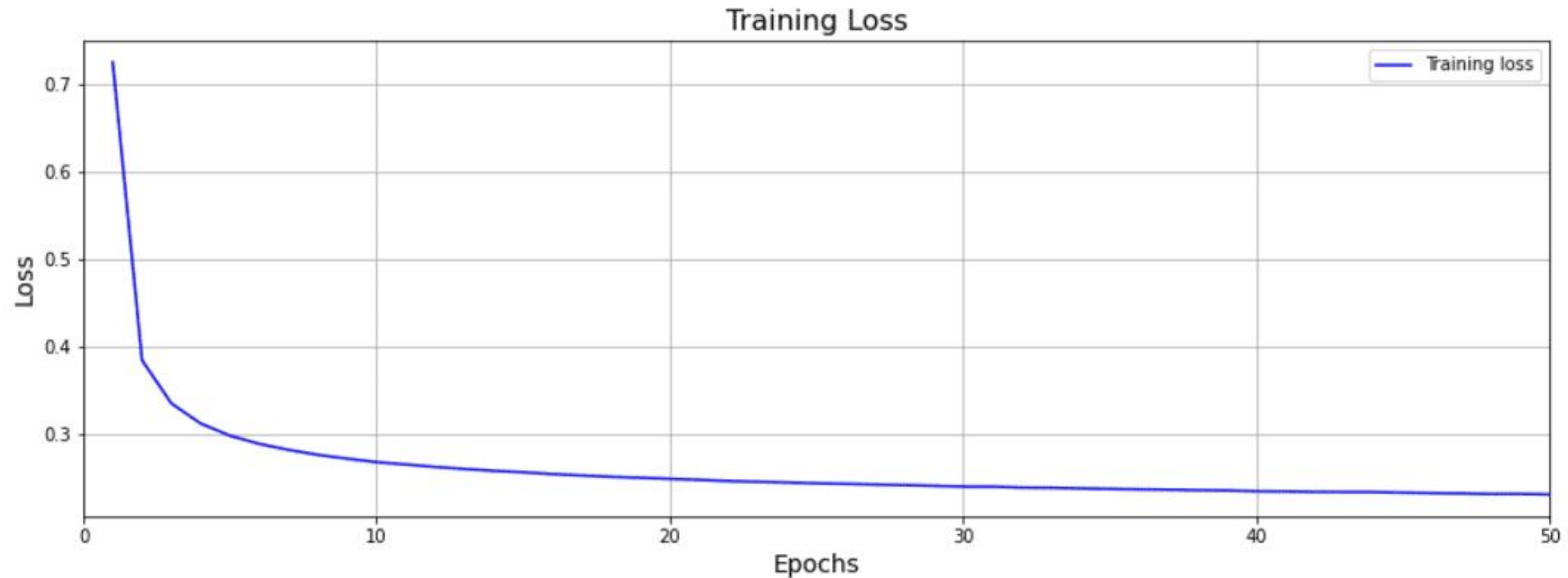


image source: <https://learnopencv.com/how-to-train-neural-networks-for-beginners/>

Underfitting and Overfitting



- Underfitting and overfitting can both lead to poor generalization performance on unseen data.

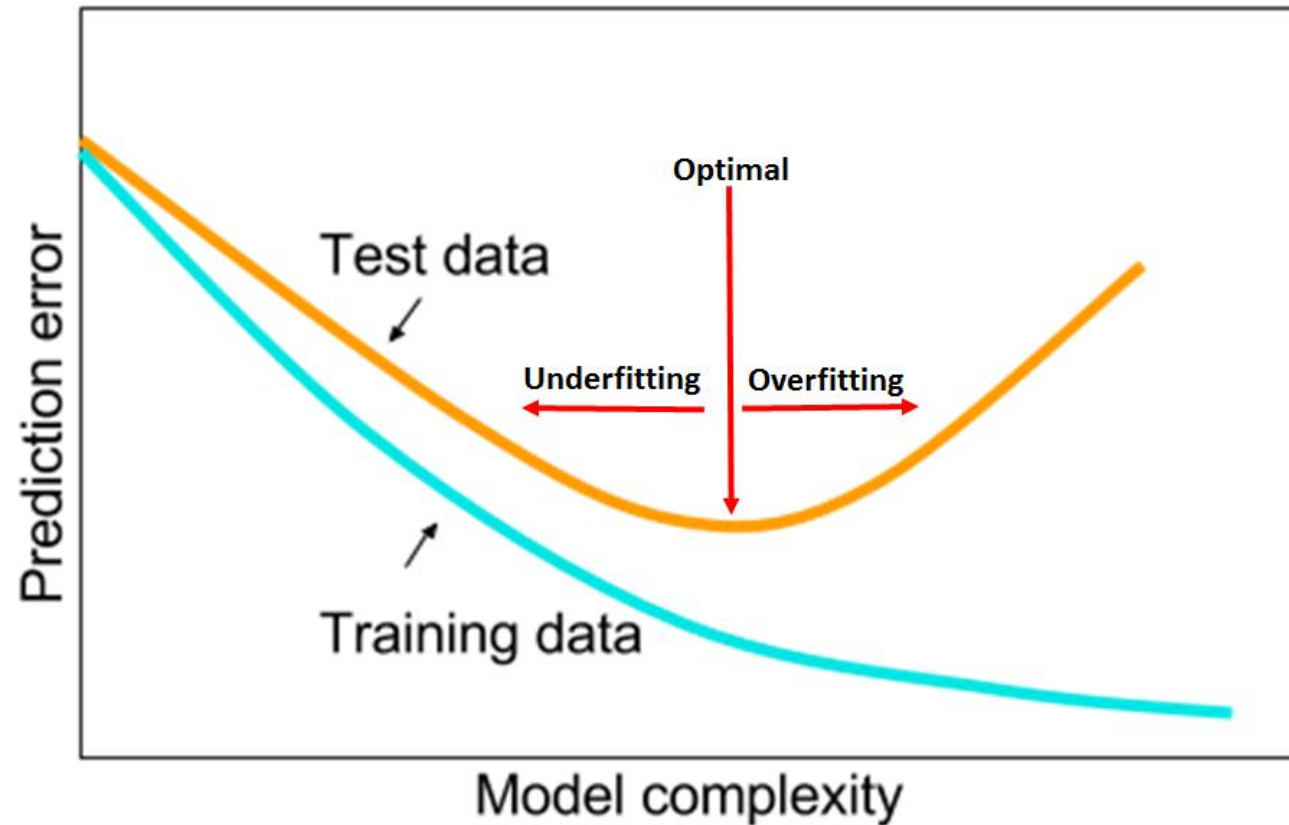


image source: <https://arxiv.org/abs/1803.09820>

Underfitting and Overfitting

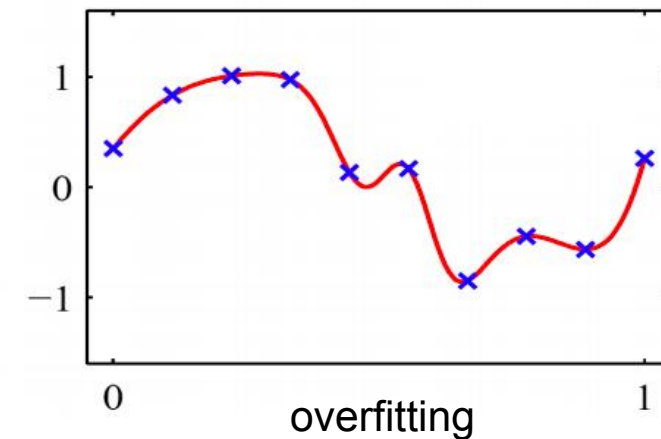
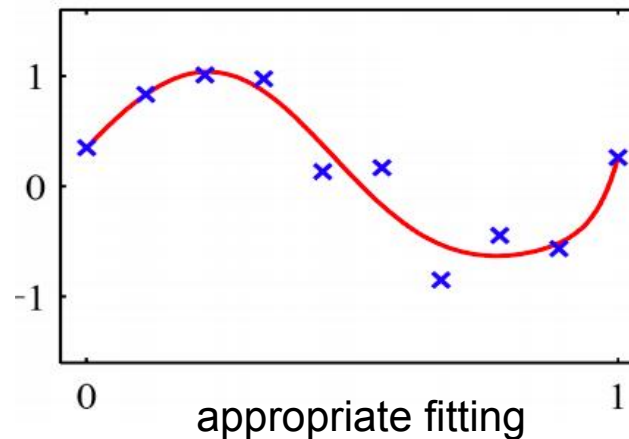
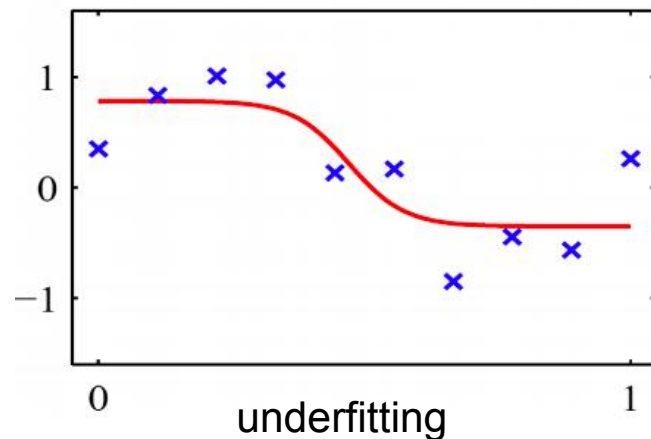


■ **Underfitting:** The model is **too simple** to capture the underlying trend of the data.

- It performs poorly on both the training set and the validation set.
- Cause: Model is not complex enough, or training was stopped too early.

■ **Overfitting:** The model is **too complex**. It memorizes the training data, including the noise, instead of learning the general pattern.

- It performs great on the training set but terribly on the validation set.
- Cause: Model is too complex for the amount of data, or training went on for too long.



underfitting and overfitting



■ To detect underfitting and overfitting:

- Use a separate **validation set**.
- During training, **evaluate the model** on this validation set after each epoch (or at regular intervals).
- **Monitor and compare** the model's performance on both the training set and the validation set.

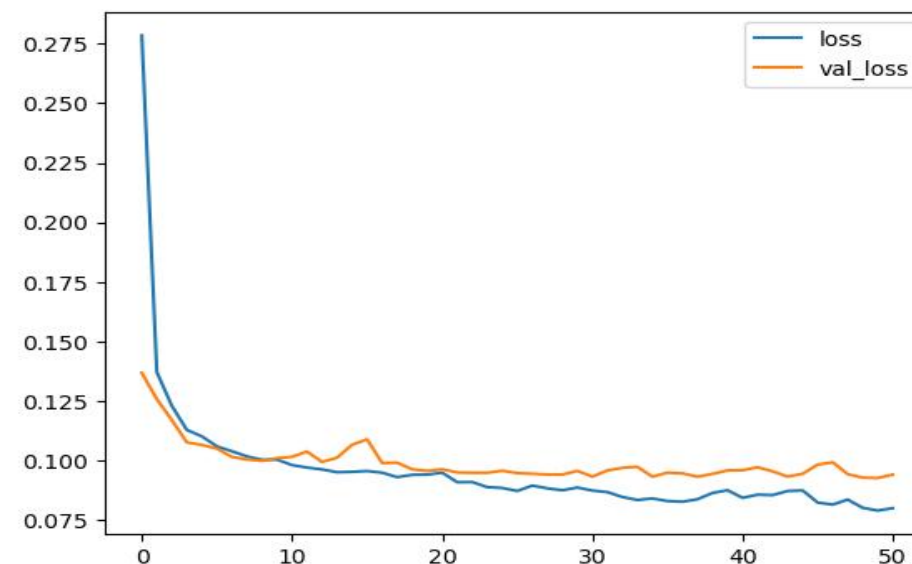
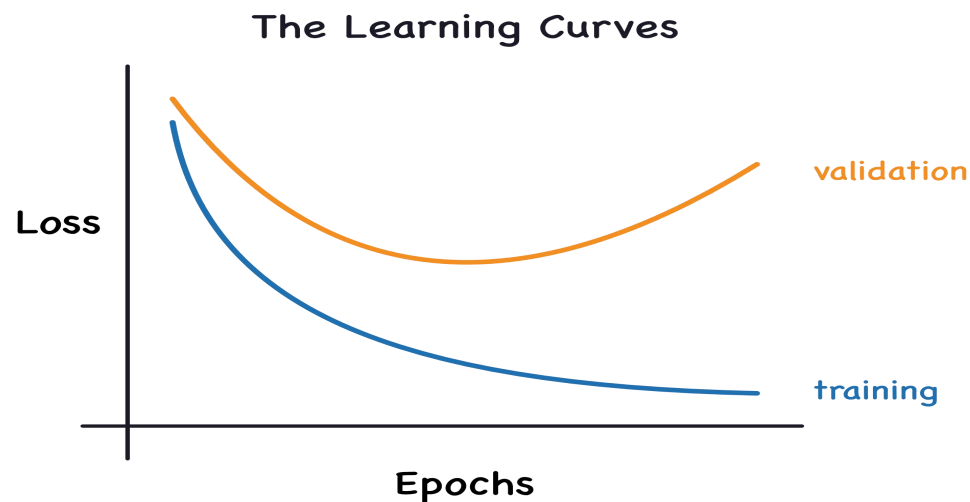


image source:

<https://www.kaggle.com/code/ryanholbrook/overfitting-and-underfitting>



Dataset Splitting

- Typically, we split a dataset into three independent sets:
 - Training set – used to update the model's weights.
 - Validation set – used to tune hyperparameters and monitor overfitting.
 - Test set – used only once at the end to evaluate final performance.

Total Data Size	Common Split Ratio (Train / Validation / Test)	Notes
1000 ~ 10,000	60 / 20 / 20 or 70 / 15 / 15	For small to medium datasets, validation and test sets need enough samples to represent the overall distribution.
10,000 ~ 100,000	80 / 10 / 10 or 90 / 5 / 5	As data size increases, you can allocate a higher percentage to training, as long as validation/test sets remain statistically meaningful.
> 1million	98 / 1 / 1 or 99 / 0.5 / 0.5 or fixed absolute numbers	For massive datasets, validation and test sets are often fixed at a certain number (e.g., 10,000 each) rather than a strict percentage.

How to Solve Underfitting



- Train longer
- Increase model complexity
 - Add More Layers
 - Add More Units/Neurons
- Architectural tweaks
 - Change Activation Functions
 - Better Weight Initialization
- Choose a more appropriate model architecture based on the task
- Reduce regularization

How to Avoid Overfitting



- Get more training data
- Apply regularization techniques
 - L1/L2 regularization (weight decay)
 - Dropout
 - Early stopping
 - Batch normalization
- Use data augmentation
 - For images: Rotate, flip, crop, zoom, add noise, or adjust brightness/contrast.
 - For text: Synonym replacement, back-translation, or random word insertion/deletion/swap.
- Reduce model complexity
 - Reduce the number of layers
 - Reduce the number of neurons/units

Outline



■ Course Logistics

- Overall Objective
- Syllabus / Projects

■ Introduction to Deep Learning

- Deep Learning and Neural Network Basics
- Training Neural Network
- Regularization



L1/L2 Regularization

$$L(w) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss: Model predictions should match training data}} + \underbrace{\lambda R(W)}_{\text{Regularization: Prevent the model from doing too well on training data}}$$

λ = regularization strength
(hyperparameter)

L2 regularization: $R(W) = \sum_l \sum_i \sum_j (w_{i,j}^{(l)})^2$

L1 regularization: $R(W) = \sum_l \sum_i \sum_j |w_{i,j}^{(l)}|$

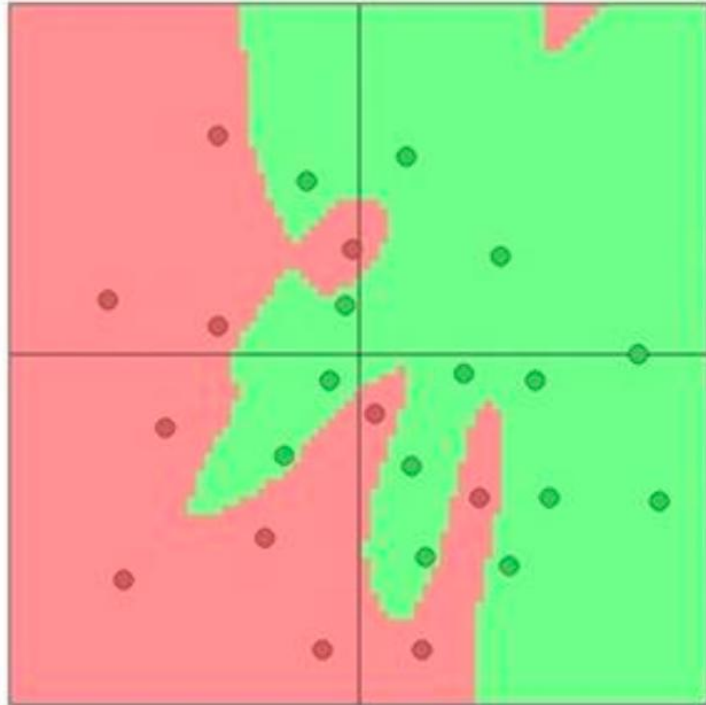
↓
rarely used

L1/L2 Regularization

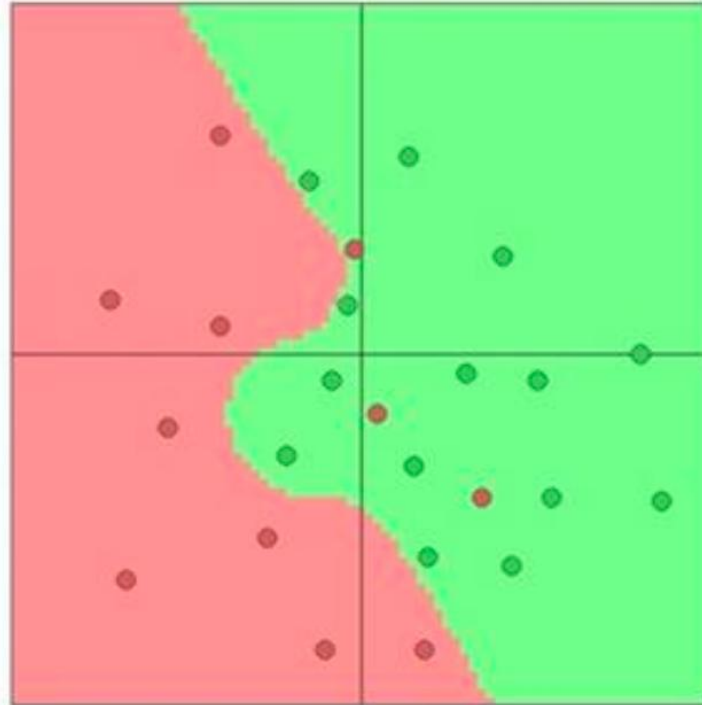


$$L(w) = \frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i) + \lambda R(W)$$

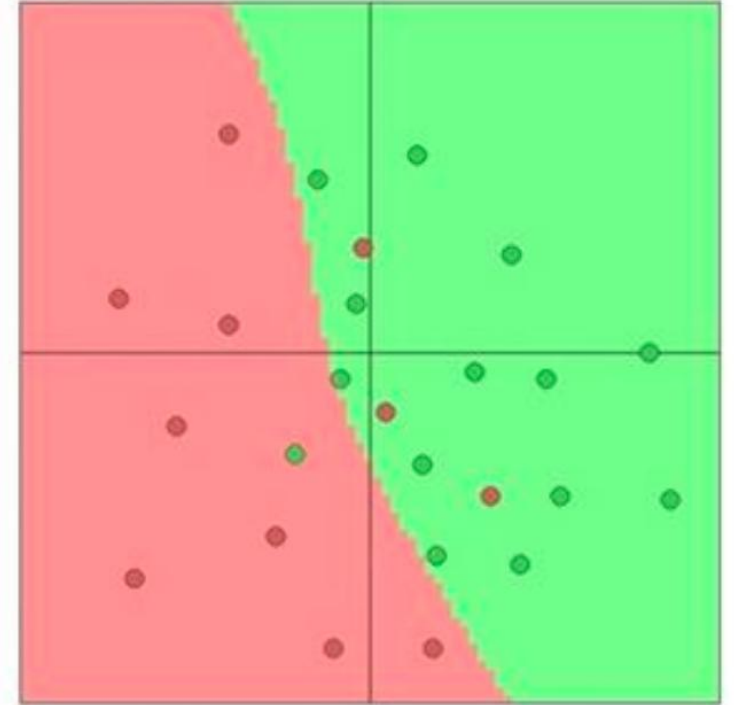
$\lambda = 0.001$



$\lambda = 0.01$



$\lambda = 0.1$



L1/L2 Regularization

■ Why use L2 regularization?

- Encourages weights to **stay small**, which can lead to **smoother, simpler** decision boundaries.
- Can help **stabilize training**.
- L2 is generally **more effective** than L1 for preventing overfitting.

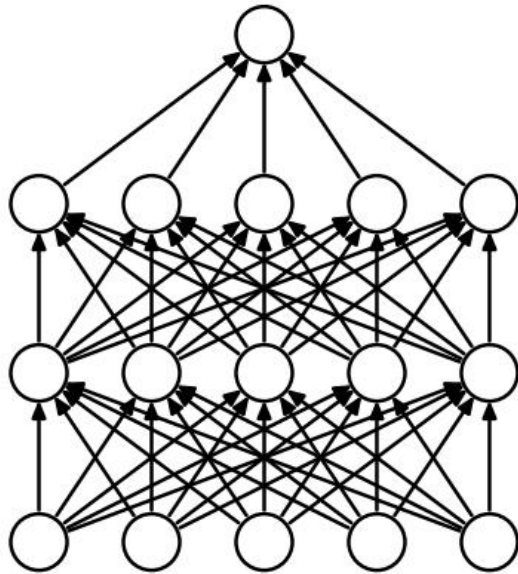
■ In practice, L2 regularization is often implemented as **weight decay** in optimizers (e.g., SGD, AdamW).

```
class AdamW(Optimizer):  
    def __init__(  
        self,  
        params: ParamsT,  
        lr: Union[float, Tensor] = 1e-3,  
        betas: Tuple[float, float] = (0.9, 0.999),  
        eps: float = 1e-8,  
        weight_decay: float = 1e-2,  
        amsgrad: bool = False,  
        *,  
        maximize: bool = False,  
        foreach: Optional[bool] = None,  
        capturable: bool = False,  
        differentiable: bool = False,  
        fused: Optional[bool] = None,  
    ):
```

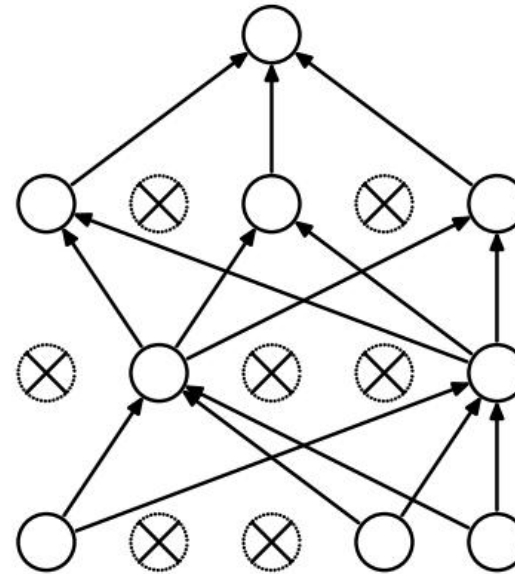
Regularization: Dropout



- In each forward pass, randomly set some neurons to zero.
- Probability of dropping is a hyperparameter; 0.5 is common.



(a) Standard Neural Net



(b) After applying dropout.

Srivastava et al, "[Dropout: A simple way to prevent neural networks from overfitting](#)", JMLR 2014

Regularization: Dropout



■ Training Phase

1. Generate a **binary mask** of the same shape as the input, where each element is 0 with probability p (drop) and 1 with probability $(1 - p)$ (keep).
2. Multiply the input by this mask to zero out dropped neurons.
3. Scale the result by $1 / (1 - p)$ to maintain the expected value (Inverted Dropout)

$$\text{output} = \frac{\text{input} * \text{mask}}{1 - p}$$

output at inference time
= expected output at training time

■ Inference Phase

- During inference, dropout is turned off: all neurons are active
- Use the full network

Regularization: Dropout



■ Code

```
import numpy as np

def dropout(x, p, training=True):
    """
    Apply dropout to input x.

    Args:
        x (np.ndarray): Input array (e.g., activations from a layer).
        p (float): Dropout probability (fraction of neurons to drop, 0 <= p < 1).
        training (bool): If False, no dropout is applied (inference mode).

    Returns:
        np.ndarray: Output with dropout applied (scaled during training).
    """
    if p < 0 or p >= 1:
        raise ValueError("Dropout probability p must be in [0, 1)")

    if not training:
        return x # No dropout during inference

    # Create a binary mask: 1 with prob (1-p), 0 with prob p
    mask = np.random.rand(*x.shape) > p

    # Apply mask and scale by 1/(1-p) to maintain expected value
    out = x * mask / (1.0 - p)

    return out
```


Regularization: Dropout



■ In Pytorch

- `model.train()`: sets the model to training mode (dropout active).
- `model.eval()`: sets the model to evaluation mode (dropout inactive).

```
model = ThreeLayerMLP(input_size, hidden_size1, hidden_size2, output_size).to(device)
criterion = nn.CrossEntropyLoss() # Define the loss function
optimizer = AdamW(model.parameters(), lr=learning_rate) # Initialize the optimizer

model.train() # Set the model to training mode
for epoch in range(epochs):
    for batch_x, batch_y in train_loader:
        batch_x, batch_y = batch_x.to(device), batch_y.to(device)

        optimizer.zero_grad() # Clear gradients from the previous step
        # Forward pass
        logits = model(batch_x) # compute predicted logits
        loss = criterion(logits, batch_y) # Compute the loss between predictions and true labels
        # Backward pass
        loss.backward() # compute gradients of the loss with respect to model parameters
        optimizer.step() # Update model parameters using the optimizer
```

Regularization: Dropout



Common Setting

- A common default value used in practice is **0.5** for **hidden layers**, while lower values like **0.2** may be used for **input layers**.
- In **Transformer-based** models like BERT and GPT, dropout rates are typically set lower than in smaller networks, usually in the range of **0.1 to 0.2**.
- In many **LLMs** such as Llama 3 and Qwen, dropout is typically **disabled** (i.e., set to 0).